

# 2026 首届 openvela AI 硬件开发者大赛 · 技术报告

## FocusLoop | 把零散的时光，织成记忆的回响。

博丽灵梦赛高 · 龚城立      版本：2026-09-16

### 1、信息表

| 项目   | 内容                      |
|------|-------------------------|
| 作品名称 | FocusLoop（腕上主动学习闭环）     |
| 队伍名称 | 博丽灵梦赛高（253）             |
| 团队分工 | 龚城立：产品定义、系统集成、测试验收与作品展示 |
| 选题方向 | 手表应用创新；AI 硬件产品创新        |

### 2、摘要

FocusLoop 面向语言学习、认证备考和技术知识复习，把用户的学习材料转成适合手表回答的短题。应用运行于 openvela QuickApp，通过 ai\_agent 调用 MiMo 生成题卡，并用本地代码完成判分、复习间隔计算与进度保存。用户主动开启智能推荐后，应用结合复习到期时间、实时压力、活动状态和反馈选择提醒时机。当前已完成五页应用、自定义 Skill、材料导入和语音控制桥。2026 年 9 月 16 日复测通过 28 项测试，连续 20 轮共 560 项检查，5 次构建均成功。语音识别和健康接口的真机运行仍待验收。

### 3、正文

#### 3.1 绪论

##### 项目背景与问题定义

备考资料常散落在笔记和文档里，固定提醒又容易在忙碌时打断用户。FocusLoop 把短时回忆放到手腕上：用户提供材料，专注后回答短题，系统安排下次复习；开启智能推荐后，再结合当前压力与活动状态选择提示时机。

##### 技术难点

三个关键问题是：把模型文本转成适配表盘的题卡；让答题进度、重启恢复与 Agent 定时任务一致；在网络、语音或健康接口异常时保留基本学习流程，并给出明确状态。

##### 创新点

导入题卡附原文依据，推荐同时检查到期与情境，用户反馈用于调整个人压力阈值。项目将常见的间隔重复方法与材料转卡、设备信号和持久定时组合，让内容生成后能继续发生复习。记忆保持率和提醒接受率是否改善，仍待用户对照实验验证。

## 3.2 系统方案设计

### 系统总体架构

系统分为输入、模型、本地状态和设备执行四部分。主题、预设内容或本地材料经 system.velaclaw 进入 ai\_agent；MiMo 返回题卡后，QuickApp 检查字段并保存。判分、间隔计算及情境判断在设备端执行；定时任务交给 Agent 的 cron\_add 和 launch\_quickapp。压力和加速度只参与本地推荐计算，不发送给模型。

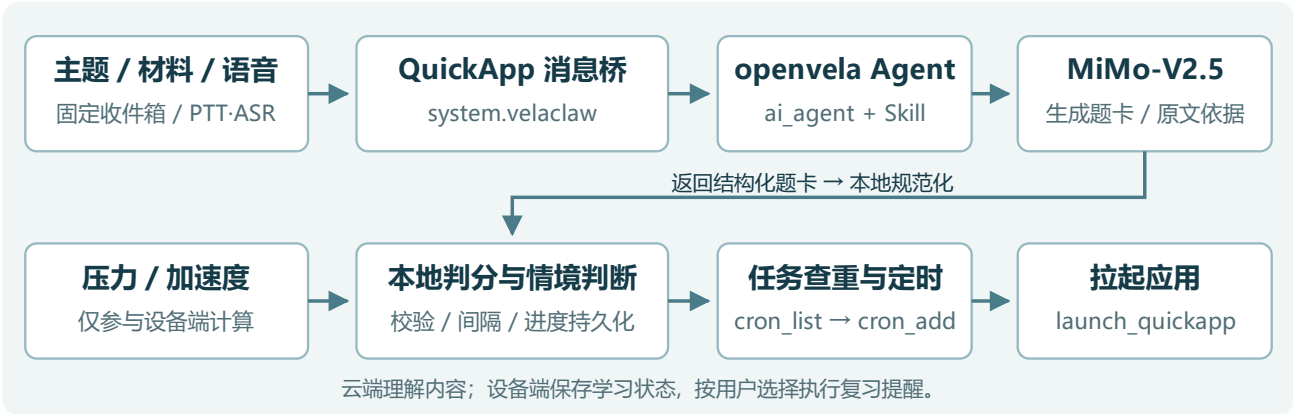


图 1 内容生成、端侧计算与 Agent 执行的数据流。

### 方案论证与选型

选择 QuickApp 是为了复用 openvela 的图形界面、存储、传感器及 Agent 通道，并在 466×466 圆形表盘上验证交互。当前运行验证采用 xiaomi\_watch\_s1 Goldfish 模拟器，没有新增硬件平台或驱动。固定词库便于离线运行，但难以覆盖用户自己的资料；纯聊天界面擅长问答，但需要用户反复组织输入。因此采用云端模型生成内容、端侧规则维护学习状态的组合。网络失败或模型输出无效时使用内置计划；健康服务缺失时停用情境推荐，时间型复习和基础学习流程仍可使用。

### 关键模块设计

一次答题需要同时更新 totalAnswers、correctAnswers 和题卡的 dueAt。调度函数保持为纯函数，便于用固定时间测试；存储和设备订阅放在独立模块。压力回调更新最近压力值，加速度回调保留最近 12 个样本，再重新计算推荐条件。页面通过订阅监测器快照显示“等待数据”“活动”或“适合复习”等状态。下表列出主要模块和源码入口。

| 模块    | 输入与输出            | 源码入口                                              |
|-------|------------------|---------------------------------------------------|
| 内容规范化 | 模型文本 → 题卡对象      | agent_protocol.js: extractPayload / normalizePlan |
| 间隔调度  | 答题结果 → dueAt     | scheduler.js: gradeCard / nextDueCard             |
| 情境判断  | 压力、采样、到期 → ready | context.js: evaluateReviewWindow                  |
| 事件与存储 | 设备订阅、偏好 → 本地状态   | context_monitor.js / store.js                     |

上述文件均位于 quickapp/focusloop/src/common/。

### 3.3 核心算法与技术原理

#### AI 算法实现

MiMo-V2.5 经 ai\_agent 调用，接口为 OpenAI 兼容的 /v1/chat/completions，使用 Authorization: Bearer 鉴权。设备端负责题卡处理，不部署大语言模型。CREATE\_PLAN 请求按主题生成 3–8 张三选一题卡；IMPORT 从固定收件箱读取材料，将知识点、问题、干扰项、解释及依据压缩成短题。

返回结果放在 <focusloop-json> 标签中。紧凑卡片格式为 [题目,正确项,干扰项 1,干扰项 2,解释,知识点,原文依据]，本地映射为 options 和 answerIndex=0。extractPayload 优先取标签内容，也可提取引号和转义边界内配平的 JSON 对象；normalizePlan 最多保留 8 张卡，补齐默认字段并处理重复 ID。题干、选项、解释、依据分别限制为 72、28、84、52 字符，避免腕上排版失控。

生成主题的请求超时为 25 秒，材料导入为 60 秒。这是配置的等待上限；到达上限或解析异常后回退到内置学习策略题，不代表已从原材料生成成功。语音入口通过 [AGENT:VOICE\_START] 与 [AGENT:VOICE\_STOP] 控制 PTT/ASR；应用收到识别文本后继续使用同一题卡生成链路。

#### 关键机制设计

复习调度：答错后 10 分钟重试；首次答对后间隔 1 天，第二次为 3 天，随后按上一间隔的 2.2 倍取整，最大 30 天。优先展示最早到期题卡。

情境判断：用户已开启智能推荐并完成首次学习；题卡已经到期或将在 30 分钟内到期；压力不高于个人阈值；至少有 6 个有效加速度样本；距离上次推荐不少于 30 分钟。任一条件不满足就不推荐。静止判定取最近最多 12 个样本，相邻三轴差的欧氏距离均值不大于 0.35；该阈值沿用设备采样值尺度，仍需真机校准。

反馈更新：压力阈值初始为 25，范围 15–35。接受时目标值取当前阈值与“样本压力 + 2”的较大值，稍后时取当前阈值与“样本压力 - 2”的较小值；新阈值为旧值的 80% 加目标值的 20%，限幅后取整。这是固定规则调整，不属于训练完成的用户行为模型。

状态恢复：下一次复习先保存到 pendingScheduleAt；Agent 使用 cron\_list 去重并创建任务，应用收到有效的 scheduled:true 回执后才清除待同步状态。失败时保留记录，首页再次尝试同步。

计算示例：已开启推荐、完成首次学习，题卡将在 20 分钟后到期，静止采样充足且冷却结束。初始压力阈值为 25。

| 输入或操作    | 结果       | 计算依据                                                                       |
|----------|----------|----------------------------------------------------------------------------|
| 当前压力 22  | 进入可推荐状态  | $22 \leq 25$ ，其余条件均满足                                                      |
| 此时选择“稍后” | 阈值更新为 24 | 目标值 $\min(25, 22-2)=20$ ； $\text{round}(25 \times 0.8 + 20 \times 0.2)=24$ |

#### openvela 系统能力的深度运用

图形能力用于五页 QuickApp 界面、圆形表盘布局与答题反馈；AI 能力通过 system.velaclaw、ai\_agent 和自定义 Skill 完成题卡生成及定时工具调用。两个补丁分别增加简短文本响应路径和 QuickApp 语音控制桥，构建脚本可重复应用。语音桥已完成 Goldfish 编译与链接，当前不将此结果视为端到端语音验收。对平台的后续建议是统一 Agent 任务回执、提供材料选择入口，并明确健康订阅在后台的生命周期。

### 3.4 系统实现

#### 软件 / 固件架构

五页 QuickApp 复用内容、调度、情境和存储模块；Agent 的导入、复习和定时操作由 Skill 约束。源码位于 quickapp/focusloop/，系统补丁位于 patches/，构建部署脚本位于 scripts/。项目采用 Apache-2.0 许可。

#### 数据流与关键流程设计

创建计划后，用户进行一次专注，再完成短文回答。答题结果立即写入本地存储，更新下一张题卡的到期时间，并提交待同步定时任务。时间型提醒由 Agent 执行；情境推荐只在用户开启功能且应用进程存活时判断。进程被系统结束后，时间型任务继续由 Agent 管理，情境监测在应用再次运行时恢复。

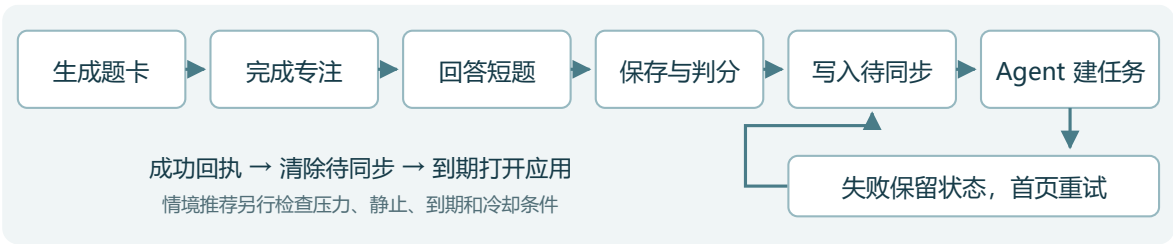


图 2 学习进度先保存，定时任务失败后可重试。

#### 硬件设计与适配

本作品为手表 QuickApp，没有新增平台、驱动、外接电路或 BOM。参赛者未领取大赛开发板，无真机照片或串口日志；运行证据为 xiaomi\_watch\_s1 Goldfish 模拟器截图与录屏。真实手表的内存、功耗、语音识别和压力订阅尚待测量。

#### 应用 / 交互端设计

界面围绕 466×466 圆形表盘设计，每页突出一个主要动作：首页查看进度和到期题卡；计划页选择主题、语音或材料导入；专注页提供 15、25、40 分钟计时；答题页展示三选一题目及解释；智能窗口提供开启、立即复习和稍后选项。材料导入当前通过 ADB 将不超过 64 KB 的 .txt 或 .md 文件送入固定收件箱，尚未提供面向普通用户的文件选择器。



图 3 Goldfish 实际运行画面：学习首页、专注、答题、结果。

#### 自定义 Skill

agent\_skills/focusloop.md 按 [FOCUSLOOP:IMPORT]、[FOCUSLOOP:REVIEW]、[FOCUSLOOP:SCHEDULE] 触发：导入只读固定材料，复习保存进度，调度查重后创建任务。当前部署位置为 /data/ai\_agent/skills/review-scheduler.md；FOCUSLOOP\_AGENT\_DATA\_DIR 可配置目录，须与 Agent 实际数据路径一致。另有 focusloop-verify 开发 Skill 检查测试、构建和制品。

### 3.5 系统测试与结果分析（务必提供量化数据）

#### 测试环境

2026 年 9 月 16 日在 Windows 11、Node.js 24.14.1、AloT Toolkit 2.0.5 上重跑仓内测试与构建。被测应用源码与主办方分支提交 64c703c 一致。历史设备联调环境为 Ubuntu 22.04 下的 xiaomi\_watch\_s1 / arm64 Goldfish；模拟器核心页面截图位于 docs/screenshots/。本轮未重新测量模型服务、语音识别或健康传感器。

#### 功能测试

本轮 28 项自动化测试覆盖 JSON 提取、字段规范化、提示词边界、间隔调度、材料导入约束、页面导航、异步回调、语音控制结果、情境判断及可选振动能力。部分测试通过读取源码检查集成约束，部分直接调用纯函数；这些测试不等同于真实设备上的传感器和语音操作。此前 Goldfish 已验证页面导航、离线计划、专注、答题和进度保存；截图与演示视频提供流程证据。

#### 性能测试

本轮保留测试和构建的逐次输出、命令、退出码与用时。计时范围为宿主机子进程的完整执行时间，不能作为模型响应速度或设备运行性能。参赛 RPK 为 72,703 字节，SHA-256 由仓库 artifacts/ 文件校验。模型延迟、ASR 准确率、真实手表内存与功耗均未测量，暂不提供数值结论。

| 测试项     | 本轮结果                  | 证据 / 测量范围                          |
|---------|-----------------------|------------------------------------|
| 功能与集成约束 | 28 项 × 20 轮 = 560 项通过 | test-01.txt 至 test-20.txt；纯函数和源码约束 |
| 测试进程耗时  | 平均 63 ms；最大 79 ms     | 含 Node 进程启动，非设备耗时                  |
| 连续构建    | 5 / 5 成功；平均 3839 ms   | build-1.txt 至 build-5.txt；宿主机构建    |
| 生产依赖审计  | 0 个漏洞                 | audit-production.txt；不含开发依赖        |
| 参赛 RPK  | 72,703 字节             | artifacts/ 中原参赛制品与 SHA-256         |
| 设备性能    | 未测量                   | 模型延迟、功耗、ASR 准确率、内存                 |

证据目录：docs/verification/2026-09-16/。summary.json 记录命令、退出码、用时及输出哈希；复测命令：node scripts/collect\_verification.cjs。

#### 可靠性与稳定性测试

连续执行 20 轮现有测试，共 560 项检查，全部通过；连续 5 次 QuickApp 构建全部成功。测试验证了坏格式回退、冷却期重新检查条件、进度先保存后同步以及振动缺失时继续运行。生产依赖审计使用 npm audit --omit=dev，结果为 0；范围不包含开发依赖。当前没有持续运行时长、设备内存波动、情境误报率或漏报率的实测数据。后续需在兼容手表上采集接受率、误打扰率与功耗，评估推荐规则是否适合实际使用。



### 3.6 AI-Native 开发说明

| 指标             | 数据                                                  |
|----------------|-----------------------------------------------------|
| AI Coding 代码占比 | 约 50%。参赛者按新增及修改代码估算，未作逐行归因统计。                       |
| 使用的 AI 工具      | Codex、Claude Code (MiMo)。DeepSeek 辅助本次报告润色。         |
| MCP 工具使用情况     | 未使用 VelaJS MCP；开发与联调使用 Shell、ADB 及官方文档。             |
| Skills 使用与新增情况 | 自建运行时 FocusLoop Skill，以及开发侧 focusloop-verify Skill。 |
| Token 使用总量     | MiMo：本项目用量 2,193,625,712+ Token；GPT 用量未单独统计。        |

#### AI 工具的作用与开发中遇到的问题

AI 辅助完成了模块实现、测试用例补充、构建排错和文档整理，参赛者负责需求取舍、集成验证及结果核对。开发中暴露的问题主要是模型返回格式不稳定和设备接口差异：前者通过本地解析、字段检查及超时回退处理；后者采用能力检查和分功能降级。项目没有记录可用于对照的纯人工开发耗时，因此不报告效率提升百分比。

AI Coding 日志按大赛手册导出至仓内 logs/；手册要求含凭证的会话整段排除，故只收录可安全公开的会话。

### 3.7 总结与展望

#### 成果总结

FocusLoop 已实现从学习材料输入、短题生成、专注、答题到下次复习安排的应用流程，并提供可安装 RPK、自定义 Skill、系统补丁和构建脚本。当前最扎实的结果是可复现的本地逻辑、异常处理和模拟器学习流程。语音与健康接口具备实现基础，真实设备效果仍需单独验收。

#### 应用前景与商业价值

首批用户定位于有持续复习需求的学生、认证备考者和技术从业者，使用场景是课间、休息或工作间隙的短时回忆。个人版可从自带材料的学习工具起步；若真实使用数据支持，再面向课程机构和企业培训提供内容管理服务。当前没有用户规模、留存或收入数据。商业判断将依据题卡质量、复习完成率、提醒接受率和设备部署成本，而非仅凭生成内容的数量。

#### 不足与未来工作

下一步先完成兼容手表上的压力订阅与语音识别验收，记录内存、功耗、识别成功率和端到端耗时；随后开展材料题卡抽检及小规模试用，比较固定提醒与情境提醒的接受情况。材料选择、跨设备课程同步和睡眠场景提醒属于后续方向。当前尚未接入小米运动睡眠历史，也未完成与米家设备的联动验证。

项目仓库：[https://github.com/open-vela/contest2026\\_253\\_bolilingmengsaigao](https://github.com/open-vela/contest2026_253_bolilingmengsaigao)

目标分支：dev-ai-contest-2026。报告依据：官方《作品提交模板》1、2、3.1–3.7。